ByteFlow: A Comparative Platform for Classical and Quantum Max-Cut Solutions

Christiaan Gouws
4332981
4332981@myuwc.ac.za

Noah Mphahlele
4323227
4323227@myuwc.ac.za

Refiloe Angela Mshiyo
4388044
4388044@myuwc.ac.za

Yolisa Mzwakali
4395423
4395423@myuwc.ac.za

Ntsekiseng Elliot Nyembe
4101764
4101764@myuwc.ac.za

Jude Joshua
4368679
4368679@myuwc.ac.za

ABSTRACT

ByteFlow is an interactive, web-based platform designed to bridge usability and educational gaps in quantum computing by enabling side-by-side benchmarking of classical and quantum optimisation algorithms on the Max-Cut problem. Synthesised from published literature, user surveys, and existing tool evaluations, ByteFlow addresses common barriers in quantum software workflows—such as steep learning curves, the absence of integrated classical baselines, and execution delays on cloud quantum hardware. Structured around a "virtual laboratory" mental model for students and researchers, the system focuses on smaller graphs—roughly single-digit to low double-digit node counts preferred for demonstration and learning scenarios—allowing users to explore problem instances without requiring deep underlying mathematical knowledge. Architecturally, ByteFlow integrates a classical solver module (executing standard Semidefinite Programming (SDP) algorithms) alongside a Quantum Approximate Optimisation Algorithm (QAOA) engine connected to IBM Quantum Cloud hardware via Qiskit Runtime sessions. To overcome remote queue timeouts and Application Programming Interface (API) authentication errors, the platform features an automated fallback handling module that transparently redirects execution to a local Qiskit Aer simulator with clear provenance logging. ByteFlow provides a comparative analytics dashboard that renders side-by-side metric visualisations—including graph partition layouts, wall-clock execution times, approximation ratios, and solution probabilities—delivering a practical, intuitive tool for evaluating quantum utility.

1 INTRODUCTION

Quantum computing is a growing area of computer science that offers new ways of solving difficult computational problems. One problem that can be used to explore this is the Max-Cut problem, where the goal is to divide the vertices of a graph into two groups while maximising the number of edges between the groups. Max-Cut can be solved using traditional classical algorithms, but quantum algorithms such as the Quantum Approximate Optimisation Algorithm provide another approach. However, learning and experimenting with quantum optimisation can be difficult for students and researchers. It often requires specialised knowledge, access to quantum hardware, and dealing with delays when using cloud-based quantum systems. Comparing classical and quantum results can also require several different tools. ByteFlow aims to address these challenges by providing an interactive web-based platform where users can create graphs, run classical and quantum algorithms, and compare their results in one easy-to-use environment.

2 USER MODEL

2.1 Data Collection

Due to time and access constraints, this research draws on secondary sources. The following findings are synthesised from published literature, official documentation, and community discussions. We synthesised findings from published literature, publicly documented tool workflows, and community discussions between early and mid-September 2026. Below are the findings that shaped the User Model, Mental Model, and Requirements Analysis.

2.1.1 Documented Observations of Developers Using Existing Quantum Tools. Published tutorials and studies of developers using IBM Quantum Composer and Qiskit notebooks reveal a steep learning curve and persistent friction. The process of converting a graph description into a runnable circuit [11], such as an adjacency list into a Hamiltonian, involves multiple non-trivial steps that are a known barrier for novice developers.

A recurring behaviour documented in the literature is long queue times on shared quantum hardware [10]. This is reflected in recent research on hybrid systems, which emphasises the need for a fallback execution to classical surrogates when quantum resources are unavailable or slow [12].

A third theme is the demand for a classical baseline. The QAOA workflow is a hybrid process in which a classical optimiser finds optimal parameters, and the final solution requires domain knowledge to "sanity-check". Consequently, developers need to compare the quantum result with a classical alternative to understand its value, a concept central to measuring "quantum utility" [5].

2.1.2 Findings from Published Interviews and Surveys. Interview and survey studies of quantum computing students, developers, and educators identify three recurring needs. First, participants consistently report the absence of a systematic way to compare classical and quantum solutions on the same problem instance, and identify this as a gap in current educational tooling [5].

Second, larger graph instances are described as interesting but impractical for live or classroom use. Smaller graphs — roughly single-digit to low double-digit node counts — are preferred for demonstration and learning scenarios [8].

Third, educators and learners emphasise the need for a mode that completes reliably within a fixed time window, even when real hardware queues are long [10]. This concern is echoed in the

broader QSE literature on hardware access variability and fallback execution.

Table 1: Review of existing quantum and classical interfaces

Tool	What worked well	What felt heavy for classroom use
1. IBM Quantum Experience / Composer [6]	Clear circuit visualiser; backend status	Long path from graph → Hamiltonian → circuit; no built-in classical baseline
2. Qiskit textbook notebooks [7]	Excellent explanations	Students must write a lot of boilerplate; comparison with classical is left as an exercise
3. Amazon Braket console [2]	Clean job history	Different SDK; not aligned with the tooling assumed by our project
4. Open-source Max-Cut visualisers (GitHub) [9]	Instant classical results and clean graph drawings	No quantum path at all
5. Online graph editors (e.g. graphonline.ru) [3]	Very fast graph input	No optimisation solvers

2.1.3 Existing-Tool Review. We reviewed publicly documented interfaces and tool workflows to identify interaction patterns relevant to our design. Table 1 summarises what each tool does well and where it falls short for classroom or demonstration use. The gap identified across all five tools is precisely the space our platform targets.

2.2 User Profiles

2.2.1 User/Researcher/Student.

- User role: End user
- Characteristics: Semi-proficient with technology, familiar with standard classical algorithms but not necessarily quantum computing. Will access and make use of software when needed for research or personal curiosity. Byteflow will be accessed with any electronic device that can access the internet with a sufficient connection. Users can be of any educational background and level.
- Primary needs/Goals: Access the "Byteflow" program and interface, view comparison of computing methods between standardized computing algorithms and quantum computing algorithms through various metrics and have an easy to use, intuitive user interface.

2.2.2 System administrator.

- User role: "Byteflow" system administrator
- Characteristics: Will monitor the systems activity daily, requires domain knowledge of user access to systems and allocation of cloud based computing resources on a network. Requires training in network administration and managing cloud computing resources.
- Primary needs/Goals: Monitor application users and computing resources to ensure optimal operations, exercising the rights to limit user access and computing resources.

2.2.3 IBM.

- User role: External system, Quantum computing resource supplier
- Characteristics: Access will be provided through Qiskit, constraints on computing resource limits are applied to operations using the resources.
- Primary needs/Goals: Serves as the provider of the IBM Quantum Cloud Computing resources for the Max Cut Problem; receives the algorithms and processes it on its servers, and returns the results to the users of the "Byteflow" interface.

2.3 UML Use Case Diagram

The use case diagram in Figure 1 models the use cases and actors involved. Unfortunately, the text in Figure 1 is small so we recommend zooming in using Ctrl + Scroll or Ctrl + Plus/Minus.

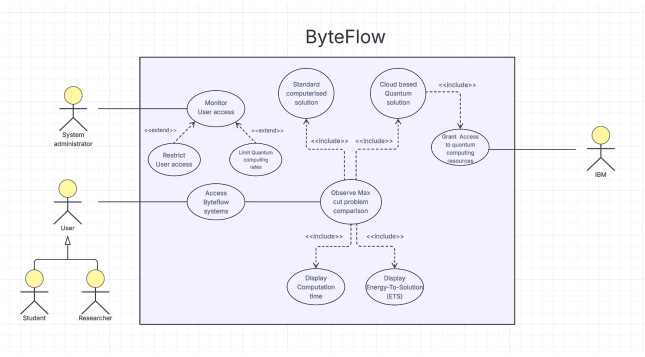


Figure 1: Use case diagram for the Max-Cut system.

3 MENTAL MODEL

3.1 System Perception

The user is expected to perceive the system as a tool for exploring or comparing different approaches to solving the same optimization problem. The application uses the Max Cut problem as an example. A Max Cut problem consists of nodes connected by edges and the objective is to divide the nodes into two groups while maximising the number or total weights of edges connecting two groups. The expected user interaction can be represented as Figure 2:

From the user's perspective, the technical processes occurring in the background should not need to be understood in order to use the application. For example, when the quantum approach is selected, the user does not need to understand the mathematical formulation of QAOA. The application will communicate the important information in an understandable form and present the resulting data visually.

The quantum component uses IBM Quantum services to provide access to quantum computing resources. From the user's perspective, IBM Quantum can therefore be viewed as a remote quantum resource used to perform the quantum part of the experiment. The application receives the resulting information and presents it to the user. The user's mental model can therefore be summarised

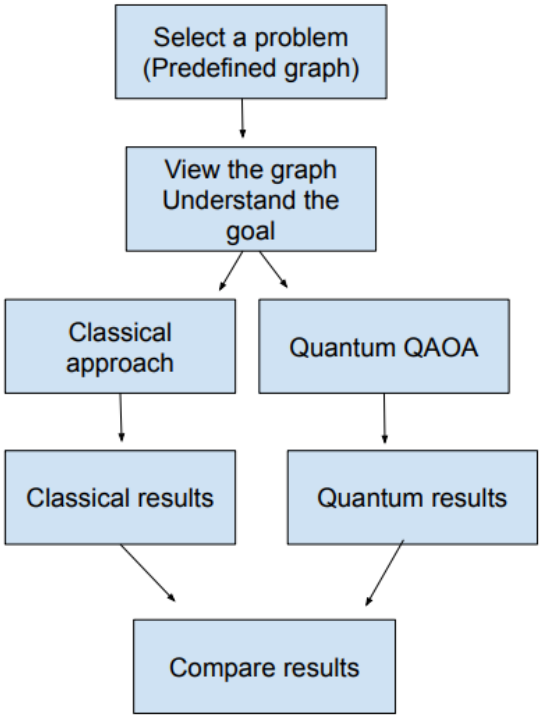


Figure 2: User Interaction diagram for the Max-Cut system.

as: Select a problem → Understand the problem → Run two approaches → Observe the results → Compare results → Learn from the comparison. This allows the application to introduce quantum computing through practical experimentation rather than requiring users to understand the underlying technology before interacting with it.

3.2 Real-World Metaphors

The application can be compared to a virtual laboratory, where users can explore and investigate problems using different computational approaches. Similar to a physical lab, where different methods can be used to investigate the same problem, the application allows users to examine a predefined Max Cut problem using both classical and quantum computing.

Selecting a predefined Max Cut problem is similar to choosing an experiment, while viewing the graph represents examining the experimental setup. The classical solver represents a traditional computational method, whereas the quantum solver represents an alternative method of investigating the same problem. The results produced by both approaches act as observations that users can examine and compare.

This metaphor helps users understand the application as an interactive learning environment where they can explore optimization problems, observe the outcomes of different computational approaches and develop a better understanding of quantum computing without needing advanced knowledge of its underlying principles.

4 REQUIREMENTS ANALYSIS

4.1 Functional & Behavioral Requirements

4.1.1 Classical Solver Module (UC1).

- The system shall execute classical Max-Cut optimisation algorithms (not custom-coded) on the predefined graph [4].
- The system shall record the classical solution outputs which includes optimal cut values and wall-clock execution times [4].

4.1.2 Quantum Execution Module (UC2).

- The system shall compile and dispatch gate-based QAOA quantum circuits to IBM Quantum Cloud hardware using sessions [4, 13].
- The system shall display the execution status and the returned measurement shot distributions from IBM Quantum processing units [4, 13].

4.1.3 Fallback Handling Module (UC3).

- The system shall detect IBM execution timeouts, session expirations, or API errors and initiate fallback procedures to a local simulator.
- The system shall notify the user of the fallback and deliver simulated QAOA expectation values without interrupting the session [1].

4.1.4 Configuration & Security Module (UC4).

- The system shall provide secure and encrypted configuration and storage for user API tokens
- The system shall validate API token credentials and then verify QPU availability before dispatching.

4.1.5 Comparative Analytics Module (UC5).

- The system shall display comparative analytics, rendering side-by-side metrics for classical vs quantum solvers [1]. These include approximation ratios, wall-clock runtimes, etc [4, 13].

4.1.6 Scenario 1: User runs a classical Max-Cut.

(1) User selects classical solver (Goemans-Williamson SDP) for the graph → System highlights classical configuration options and enables the run button [4].
(2) User clicks the Run button → System dispatches the predefined matrix to the classical execution engine [4].
(3) System completes execution → System highlights the cut edges and displays the wall-clock execution time (metrics) [4].

4.1.7 Scenario 2: User runs a quantum Max-Cut with successful IBM execution.

(1) User selects the QAOA quantum solver, configures layer depth p, and inputs an API token → System validates the token and checks QPU backend status [4].
(2) User clicks the Run button → System constructs the QAOA circuit ansatz, transpiles it for the target IBM QPU, and initiates an IBM Runtime session [4, 13].
(3) IBM QPU returns measurement samples → System computes the quantum expectation value, extracts the highest-probability bitstring cut, and displays final metrics [4, 13].

4.1.8 Scenario 3: User runs a quantum Max-Cut that times out and falls back.

(1) User initiates QAOA execution (same as scenario 2) → System submits the job session and displays initial queue status.
(2) IBM Quantum Cloud backend exceeds the queue time-to-live threshold → System flags a remote execution timeout exception and triggers fallback handling to local Qiskit Aer Sim [1].
(3) Local sim completes → System renders simulated Max-Cut results clearly annotated with a local sim provenance label [1].

4.1.9 Scenario 4: User views the comparatitive dashboard.

(1) User navigates to the Comparative Analytics tab/dashboard after running both solvers → System fetches stored execution metrics [1].
(2) User selects specific classical and quantum run instances to benchmark → System renders dual metric visualisations displaying classical vs quantum cut partitions side-by-side [1].
(3) System compiles comparative metrics → System generates side-by-side charts comparing approximation ratios, solution probabilities, and execution runtimes [4, 13].

4.1.10 State Diagram. The state diagram in Figure 3 models the solver execution life cycle. Unfortunately, the text in Figure 3 is small so we recommend zooming in using Ctrl + Scroll or Ctrl + Plus/Minus.

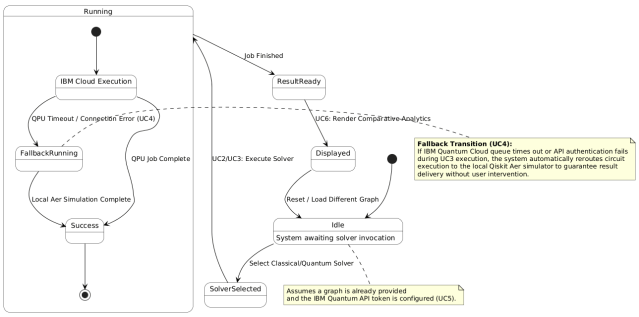


Figure 3: State diagram for the Max-Cut system.

4.2 Use Case Specifications

This section details the specific workflows of the system. The foundational process, UC1 (Execute Classical Max-Cut Solver), is first outlined in Table 2.

Table 2: Use Case UC1 — Execute Classical Max-Cut Solver

Field	Value
ID	UC1
Actor	User
Goal	Execute a classical optimisation algorithm on the loaded graph to compute baseline Max-Cut results.
Preconditions	A graph is loaded in the active workspace.
Trigger	User selects a classical algorithm and initiates execution.
Main Flow	1. System dispatches the graph adjacency matrix to the classical execution engine. 2. System computes the maximum cut partition, optimal cut value, and wall-clock runtime. 3. System displays the classical partition layout and stores solution metrics.
Alternate Flows	A1. Classical execution error → System displays an error message and prompts retry.
Postconditions	Classical Max-Cut metrics are saved for comparative analytics.
Related	None

The detailed specifications for the remaining use cases are provided in the following tables. Table 3 outlines the execution flow for UC2 (Quantum Max-Cut Solver), while Table 4 describes the automatic fallback procedures for UC3. The secure configuration of credentials is detailed in Table 5 (UC4), and the system's benchmarking capabilities are outlined in Table 6 (UC5).

Table 3: Use Case UC2 — Execute Quantum Max-Cut Solver (QAOA)

Field	Value
ID	UC2
Actor	User
Goal	Run a QAOA circuit on IBM Quantum Cloud hardware to obtain quantum Max-Cut results.
Preconditions	A graph is loaded and an IBM Quantum API token is configured (UC4).
Trigger	User configures QAOA parameters and initiates quantum execution.
Main Flow	1. System converts the graph to an Ising Spin Hamiltonian and builds the QAOA circuit ansatz. 2. System initiates an IBM Quantum Runtime session and submits variational execution jobs. 3. System streams parameter updates and renders QPU measurement shot distributions.
Alternate Flows	A1. IBM queue timeout or authentication failure → System triggers fallback to the local simulator per UC3.
Postconditions	Quantum Max-Cut results and parameter trajectories are stored for comparative analysis.
Related	≪include≫ UC4 (Configure IBM Quantum API Token)

Table 4: Use Case UC3 — Fallback to Local Simulator

Field	Value
ID	UC3
Actor	User
Goal	Automatically redirect QAOA execution to a local quantum simulator when remote cloud execution fails.
Preconditions	UC2 execution is active and has not yet returned a result.
Trigger	IBM Quantum Cloud queue timeout or authentication failure occurs during UC2.
Main Flow	1. System flags the remote execution failure and notifies the user of the simulation fallback. 2. System dispatches the QAOA circuit to the local Qiskit Aer simulator engine. 3. System calculates simulated expectation values and renders results with provenance labels.
Alternate Flows	A1. Simulator memory limit exceeded → System aborts the run and prompts the user to reduce graph size.
Postconditions	Simulated QAOA results are delivered without interrupting the user session.
Related	≪extend≫ UC2 (Execute Quantum Max-Cut Solver)

Table 5: Use Case UC4 — Configure IBM Quantum API Token

Field	Value
ID	UC4
Actor	User
Goal	Securely store and validate an IBM Quantum API token for remote QPU access.
Preconditions	User holds a valid IBM Quantum account credential.
Trigger	User accesses configuration settings, or UC2 detects a missing token.
Main Flow	1. User enters the IBM Quantum API token in the configuration field. 2. System validates the token and checks IBM Quantum backend availability. 3. System encrypts and stores the validated token for session use.
Alternate Flows	A1. Invalid API token → System displays an authentication error and requests valid credentials.
Postconditions	System is authenticated to dispatch execution jobs to IBM Quantum Cloud.
Related	None

Table 6: Use Case UC5 — View Comparative Analytics Dashboard

Field	Value
ID	UC5
Actor	User
Goal	Compare classical versus quantum Max-Cut results in a side-by-side visual dashboard.
Preconditions	At least one execution result exists (classical, quantum, or both).
Trigger	User navigates to the Comparative Analytics view.
Main Flow	1. System retrieves recorded execution artefacts for the active graph runs. 2. System renders side-by-side graph partitioning layouts and measurement distributions. 3. System plots comparative charts for approximation ratios, solution probabilities, and runtimes.
Alternate Flows	A1. Incomplete solver data → System displays placeholder guidance to execute the missing solver(s).
Postconditions	Comparative benchmarking visualisations and metrics are displayed.
Related	None

REFERENCES

[1] Mina Alipour. 2025. Quantum Software: From Engineering to Interaction. In *2025 IEEE Conference on Cloud and Big Data Computing (CBDCom)*. IEEE, 1–8.

[2] Amazon Web Services. 2024. Amazon Braket Console. AWS Documentation. https://aws.amazon.com/braket/

[3] Graph Online. 2024. Graph Online: Graph Editor and Visualiser. Web application. https://graphonline.ru/

[4] Gian Giacomo Guerreschi and Anne Y. Matsuura. 2019. QAOA for Max-Cut requires hundreds of qubits for quantum speed-up. *Scientific Reports* 9 (2019), 6903. doi:10.1038/s41598-019-43176-9

[5] Nils Herrmann, Daanish Arya, Florian Preis, Stefan Prestel, Marcus W. Doherty, Angus Mingare, and Jason C. Pillay. 2023. Quantum Utility – Definition and Assessment of a Practical Quantum Advantage. *arXiv preprint arXiv:2303.02138* (2023). arXiv:2303.02138 [quant-ph]

[6] IBM Quantum. 2024. IBM Quantum Composer. IBM Quantum Platform. https://quantum.ibm.com/composer

[7] IBM Quantum. 2024. Qiskit Textbook: Quantum Approximate Optimization Algorithm. Qiskit Documentation. https://qiskit.org/learn/

[8] IBM Quantum. 2024. Solving Max-Cut Problems Using QAOA with Qiskit. Qiskit Documentation. https://qiskit.org/documentation/

[9] Open-source contributors. 2024. Max-Cut Visualiser (GitHub repository). GitHub. https://github.com/

[10] Gokul Subramanian Ravi, Kaitlin N. Smith, Pranav Gokhale, and Frederic T. Chong. 2021. Quantum Computing in the Cloud: Analyzing Job and Machine Characteristics. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, Storrs, CT, USA, 39–50.

[11] Krishna Upadhyay, Vinaik Chhetri, A. B. Siddique, and Umar Farooq. 2025. Analyzing the Evolution and Maintenance of Quantum Software Repositories. In *2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, Helsinki, Finland, 173–184.

[12] Benjamin Zec, Lukas Schmidbauer, Maja Franz, and Wolfgang Mauerer. 2026. Towards Tensor-Network SAT-Solvers for Quantum-Classical Workflows. *arXiv preprint arXiv:2608.02041* (2026). arXiv:2608.02041 [quant-ph]

[13] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D. Lukin. 2020. Quantum Approximate Optimization Algorithm: Performance, Mechanism, and Implementation on Near-Term Devices. *Physical Review X* 10, 2 (2020), 021067. doi:10.1103/PhysRevX.10.021067